



Universitatea Tehnică din Cluj-Napoca

Sisteme Inteligente



Proiect Agentic AI

Code Review și Detecția Vulnerabilităților de Securitate
folosind agenți AI

Mai 2026

1 Prolog

Imaginați-vă ceva de cybersecurity... eu n-am avut inspirație aici.

2 Context general

Securitatea software a devenit una dintre provocările fundamentale ale ingineriei moderne. Conform raportului IBM *Cost of a Data Breach 2024*, costul mediu global al unei breșe de securitate a depășit 4.8 milioane de dolari, iar timpul mediu de detecție al unei vulnerabilități exploatare rămâne la 204 zile. Paradoxul este că marea majoritate a acestor vulnerabilități – SQL injection, broken authentication, dependențe cu CVE-uri cunoscute – sunt detectabile static, înainte de deployment, dacă procesul de code review este suficient de riguros.

Practica DevSecOps – integrarea securității în fiecare etapă a ciclului de dezvoltare software, nu doar la final – a câștigat adoptare largă la nivel de principiu, dar rămâne dificil de implementat consistent la nivel de echipă. Code review-ul uman, oricât de valoros, suferă de limitări inerente: oboseala cognitivă, lipsa de timp, cunoaștere incompletă a tuturor CVE-urilor relevante pentru un stack specific și inconsistența între revieweri. Sistemele statice de analiză a codului (*linters*, *SAST tools*) acoperă parțial acest gol, dar sunt rigide: nu raționează, nu explică, nu citează surse și nu se adaptează la contextul specific al unui proiect.

Proiectul de față propune construirea unui sistem multi-agent capabil să preia un repository GitHub, să analizeze codul sursă din perspectiva securității și calității, să identifice vulnerabilități raportând la baze de date CVE și ghiduri OWASP și să producă un raport structurat, cu severitate și sugestii de remediere argumentate. Elementul distinctiv al arhitecturii este **agentul cu memorie augmentată**: sistemul nu numai că analizează cod, ci memorează feedbackul primit pe review-urile anterioare și îl încorporează în analizele viitoare – un mecanism de îmbunătățire continuă ce nu există în niciun tool SAST convențional. Arhitectura se bazează pe trei piloni:

- **Retrieval-Augmented Generation (RAG)** – sistemul nu generează concluzii de securitate din memoria parametrică a LLM-ului, ci le fundamentează pe un corpus actualizat: NVD/CVE database, OWASP Top 10 și ASVS, CWE, documentația de securitate a framework-urilor utilizate (Django, FastAPI, Express) și ghiduri de stil (PEP 8, Google Style Guide).
- **Sisteme multi-agent** – sarcina complexă de code review este descompusă în agenți specializați: parsare și înțelegere a structurii codului, scanare de securitate, evaluare a calității, augmentare cu memorie episodică și generare de raport.
- **Orchestrare (LangGraph)** – agenții comunică prin intermediul unui graf de stări cu tranziții condiționale, permițând trasabilitate completă a raționamentului și o buclă de feedback care crește acoperirea în cazul fișierelor cu vulnerabilități complexe.

3 Obiective

Proiectul urmărește formarea următoarelor competențe tehnice și analitice:

- Construirea unui corpus de securitate structurat și indexarea lui semantică cu ChromaDB, pornind de la CVE-uri NVD, ghiduri OWASP, CWE și documentație de securitate a framework-urilor.
- Implementarea unui agent de parsare a codului (*Code Parser Agent*) care clonează un repository GitHub, identifică fișierele relevante, extrage structura (funcții, clase, importuri, dependențe) și produce o reprezentare formală validată cu Pydantic.
- Proiectarea și implementarea unui pipeline RAG cu LangChain care recuperează context de securitate relevant pentru fiecare fragment de cod analizat, citând CVE-ul sau ghidul OWASP specific.
- Construirea unui agent de evaluare a calității codului (*Code Quality Agent*) care calculează complexitatea ciclomatică, identifică code smells și verifică respectarea convențiilor de stil.
- Implementarea unui agent cu memorie episodică (*Self-Improving Feedback Agent*) care stochează feedbackul uman pe review-urile anterioare și îl recuperează semantic pentru a îmbunătăți review-urile viitoare.
- Implementarea unui agent de raportare (*Report Generator Agent*) care produce un raport Markdown structurat cu severitate, fragmente de cod afectate, sugestii de remediere citate din corpus și un scor global de securitate.
- Orchestrarea tuturor agenților folosind LangGraph cu stări, tranziții și condiții de oprire bine definite, inclusiv o buclă de feedback pentru fișierele cu acoperire RAG insuficientă.
- Integrarea sistemului într-o interfață grafică interactivă cu Streamlit, cu input de URL de repository, vizualizarea progresului per agent și export de raport.
- Containerizarea aplicației cu Docker pentru reproducibilitate și portabilitate.

4 Câteva aspecte despre RAG

La nivel conceptual, mecanismul fundamental din spatele unui sistem RAG (*Retrieval-Augmented Generation*) se poate sumariza simplu: injectezi un document relevant în contextul unui LLM și îl lași să răspundă bazat pe acel document, nu pe memoria sa parametrică. Aplicat la securitatea software, aceasta înseamnă că agentul nu “știe” din antrenament că `cursor.execute(f"SELECT * FROM users WHERE id=user_input")` este vulnerabil la SQL injection – știe pentru că a recuperat din vectorstore fragmentul relevant din OWASP A03:2021 sau din CWE-89, care descrie exact acest pattern. Distincția este esențială: un LLM antrenat poate halucina CVE-uri inexistente; un sistem RAG bine construit citează CVE-ul real, cu CVSS score și link la NVD.

Cu toate acestea, în implementările software din producție, inginerii se confruntă cu o problemă de scalabilitate: baza de date NVD conține peste 250.000 de CVE-uri, OWASP ASVS are sute de cerințe, iar documentația de securitate pentru un singur framework major poate depăși 1.000 de pagini. Prin urmare, un sistem RAG veritabil reprezintă un pipeline care identifică exclusiv fragmentele de text relevante pentru codul analizat și le injectează contextual în prompt.

4.1 Arhitectura Tehnică a unui Sistem RAG

O arhitectură RAG pregătită pentru medii de producție este divizată în două faze distincte: faza de încărcare și procesare a datelor (*Ingestion Phase*) și faza de inferență (*Inference Phase*).

4.1.1 Faza 1: Încărcarea și Procesarea Datelor (Ingestion)

Această etapă se execută asincron, înainte ca utilizatorul să interacționeze cu sistemul, indexând baza de cunoștințe de securitate:

1. **Curățarea Datelor:** Se extrage textul util din surse eterogene: CVE-uri în format JSON din NVD API, ghiduri OWASP în format Markdown, documentație framework-uri în format HTML sau RST, ghiduri PEP în format text. Se elimină referințele circulare, antetele redundante și metadatele irelevante pentru recuperare semantică.
2. **Segmentarea Textului (Chunking):** Documentele de securitate sunt segmentate în fragmente de câte 400–500 de tokeni, cu o suprapunere de 50–80 de tokeni. Atenție specială la CVE-uri: fiecare CVE trebuie să rămână într-un singur chunk – nu tăiați un CVE la jumătate, altfel agentul poate cita un CVSS score dintr-un CVE diferit față de descrierea vulnerabilității.
3. **Vectorizarea (Embedding):** Fiecare segment de text este transmis modelului text-embedding-3-small. Codul sursă și pseudocodul din documentația de securitate au proprietăți semantice diferite față de textul natural – testați dacă un model de embedding specializat pe cod (de ex. code-embedding-ada-002) produce scoruri de similaritate mai bune pentru interogările din codul sursă.
4. **Baza de Date Vectorială:** Vectorii rezultați sunt stocați în ChromaDB cu meta-date esențiale: source (ex. nvd:CVE-2024-1234), doc_type (cve, owasp, cwe, style_guide, framework_doc), cvss_score pentru CVE-uri, owasp_category pentru ghiduri OWASP. Fără aceste metadate, agentul de securitate nu poate filtra rezultatele după severitate.

4.1.2 Faza 2: Inferența și Fluxul de Interogare

Această fază se execută în timp real pentru fiecare fișier sau funcție analizată:

1. **Vectorizarea Interogării:** Când agentul de securitate primește un fragment de cod (de exemplu, o funcție de autentificare cu query SQL construit prin concatenare de string-uri), îl trimite modelului de embedding. Interogarea nu este fragmentul brut de cod – este o descriere semantică extrasă din cod: *“SQL query built by string concatenation with user input, no parameterized queries”*.
2. **Căutarea Vectorială (Recuperarea):** Sistemul efectuează o căutare de similitudine în vectorstore, returnând primele K fragmente (de regulă, $K \in [3, 6]$) cele mai relevante. Se poate filtra și după doc_type: pentru detecția de vulnerabilități, prioritizați CVE-uri și OWASP; pentru code smells, prioritizați ghiduri de stil și design patterns.

3. Asamblarea Contextului: Backend-ul construiește promptul final:

“Ești un expert în securitate software. Analizează fragmentul de cod de mai jos și identifică vulnerabilitățile de securitate, raportând strict la contextul furnizat. Nu inventa CVE-uri.

Corpus de securitate: [CVE-2021-44228 – Log4Shell desc.] [OWASP A03:2021 – Injection] [CWE-89 – SQL Injection]

Cod analizat: query = f"SELECT * FROM users WHERE id={uid}"

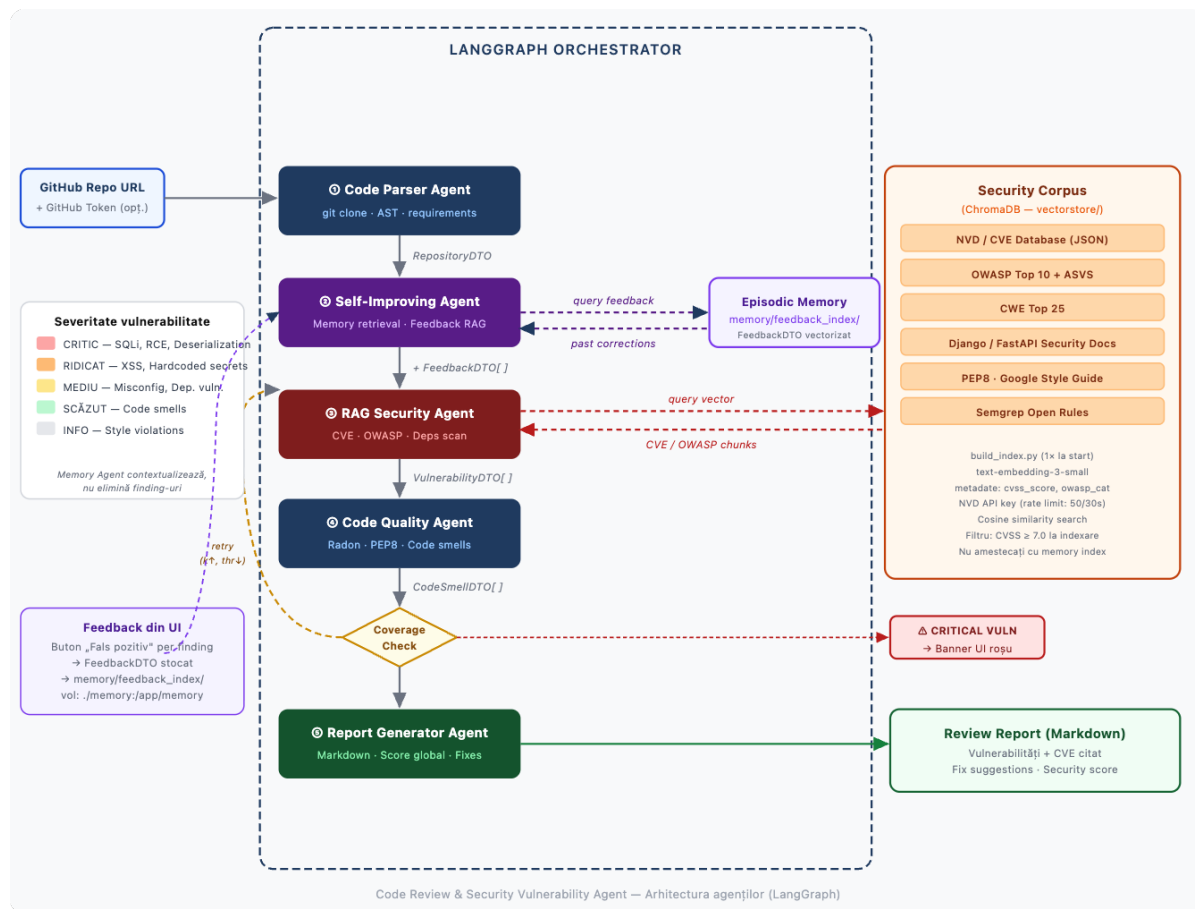
4. Generarea Răspunsului: LLM-ul analizează contextul și produce un VulnerabilityDTO structurat: tipul vulnerabilității, CVE-ul citat, fragmentul de cod afectat, linia, severitatea și sugestia de remediere – toate ancorate în corpus-ul recuperat.

4.2 Memorie Episodică – Dincolo de RAG Clasic

Sistemul extinde paradigma RAG clasică cu un al doilea vector store dedicat memoriei episodice: feedbackul uman acumulat pe review-urile anterioare. Dacă un reviewer a marcat un finding ca fals pozitiv – de exemplu, agentul a raportat o *“hard-coded secret”* pentru un string care era de fapt un placeholder din teste – această corecție se stochează ca un FeedbackDTO. La review-urile viitoare, agentul de feedback recuperează semantic corecțiile relevante și le injectează în promptul agentului de securitate. Rezultatul este un sistem care, spre deosebire de orice tool SAST static, devine mai precis pe măsură ce primește feedback.

4.3 Provocări Avansate în Implementare

- **Fals-Pozitive în Securitate:** Un model de embedding poate scora similar un fragment de cod benign față de un pattern vulnerabil dacă vocabularul este parțial suprapus. De exemplu, orice funcție care menționează password poate scora bine față de CVE-uri despre autentificare, chiar dacă codul este corect. Pragul threshold și mecanismul de feedback sunt esențiale pentru a controla rata fals-positivelor.
- **CVE-uri cu Dată de Expirare:** Un CVE patchuit într-o versiune nouă a unei librării nu mai este relevant dacă requirements.txt specifică versiunea corectă. Agentul de securitate trebuie să verifice versiunea dependenței din fișierul de configurare înainte de a raporta o vulnerabilitate bazată pe CVE.
- **Interogare Hibridă:** Căutarea pur semantică poate rata vulnerabilități identificate prin pattern matching exact (de ex. eval(user_input), exec(request.GET["cmd"])). Combinați embedding search cu căutare lexicală BM25 sau cu un set de reguli Semgrep pre-definite ca primă trecere rapidă.



Sugestie de structură, pe fișiere:

```

1 code-review-agent/
2 |
3 |-- .env                                <- secretele aplicatiei voastre
4 |-- .env.example                       <- template pentru chei API
5 |-- .gitignore                         <- include .env si vectorstore/
6 |-- Dockerfile
7 |-- docker-compose.yml
8 |-- requirements.txt
9 |-- README.md
10 |
11 |-- corpus/                            <- baza de cunostinte de securitate
    |   pentru RAG
12 |   |-- cve/                           <- CVE-uri NVD (JSON sau TXT, top-500
    |   relevante)
13 |   |-- owasp/                          <- OWASP Top 10, ASVS (Markdown)
14 |   |-- cwe/                            <- Common Weakness Enumeration (selectie)
15 |   |-- framework_docs/                <- securitate Django, FastAPI, Express
    |   etc.

```

```

16 | `-- style_guides/                <- PEP8, Google Style Guide, ESLint rules
17 |
18 | -- data/                        <- repo-uri de test + output-uri
19 | | -- repos/                    <- clone-uri locale ale repo-urilor
    | analizate
20 | | -- review_exemplu.json
21 | `-- review_exemplu_report.md
22 |
23 | -- memory/                     <- stocarea episodică a feedbackului uman
24 | | -- feedback_store.json      <- feedbackuri persistate între sesiuni
25 | `-- feedback_index/          <- vectorstore separat pentru memoria
    | episodică
26 |
27 | -- vectorstore/                <- generat de build_index.py; montat ca
    | volum Docker
28 |
29 | -- logs/
30 | | -- rag_evaluation.json
31 | | -- retrieval_heatmap.png
32 | | -- vulnerability_distribution.png
33 | | -- workflow_graph.png
34 | `-- run_<timestamp>.json
35 |
36 | -- notebooks/
37 | `-- demo_pipeline.ipynb
38 |
39 | -- scripts/
40 | | -- build_index.py            <- rulat O SINGURA DATA
41 | | -- evaluate_rag.py          <- RAGAS evaluation
42 | | -- test_parser.py
43 | `-- test_retrieval.py
44 |
45 | `-- src/
46 | | -- dtos.py                  <- toate DTO-urile si enumerile
47 | | -- app.py                   <- Streamlit UI
48 | |
49 | | -- agents/
50 | | | -- __init__.py
51 | | | -- parser_agent.py        <- CodeParserAgent
52 | | | -- security_agent.py      <- RAGSecurityAgent
53 | | | -- quality_agent.py       <- CodeQualityAgent
54 | | | -- feedback_agent.py      <- SelfImprovingFeedbackAgent
55 | | | -- report_agent.py        <- ReportGeneratorAgent
56 | |
57 | | -- tools/
58 | | | -- __init__.py
59 | | | -- repo_tools.py          <- clone_repo(), list_files(), read_file
    | | | ()
60 | | | -- vector_tools.py        <- build_index(), build_memory_index()
61 | |
62 | `-- graph/

```



```

63 | -- __init__.py
64 `-- workflow.py          <- WorkflowState + StateGraph

```

Proiectul constă în construirea unui sistem complet de code review automat. Utilizatorul furnizează URL-ul unui repository GitHub public (sau privat, cu token); sistemul clonează repo-ul, îl analizează printr-un lanț de agenți specializați și returnează un raport cu vulnerabilitățile de securitate descoperite, ierarhizate după severitate, cu fragmente de cod afectate, CVE-urile sau ghidurile OWASP citate și sugestii de remediere.

Categoriile de vulnerabilități și probleme de calitate acoperite de sistem sunt descrise în tabelul următor:

Categorie	OWASP / CWE	Severitate tipică	Exemplu
SQL / NoSQL Injection	A03:2021, CWE-89	CRITIC	f"SELECT...{uid}"
Broken Authentication	A07:2021, CWE-287	CRITIC	Parole hardcodate
Sensitive Data Exposure	A02:2021, CWE-312	RIDICAT	Chei API în sursă
Dependențe vulnerabile	A06:2021, CVE NVD	VARIABIL	django<4.2
Security Misconfiguration	A05:2021, CWE-16	MEDIU	DEBUG=True
Cross-Site Scripting (XSS)	A03:2021, CWE-79	RIDICAT	innerHTML=input
Deserialization nesigură	A08:2021, CWE-502	CRITIC	pickle.loads()
Code smells / Complexitate	PEP8, Radon	SCĂZUT	Complexitate ciclom. > 10

Tabela 1: Categoriile de vulnerabilități și probleme de calitate analizate de sistem.

Inițializarea mediului virtual se face cu `uv init .`, iar dependențele se adaugă cu `uv add <pachet>`. Cheile API – pentru OpenAI și, opțional, pentru GitHub (necesar dacă analizați repository-uri private) – se configurează în fișierul `.env`.

Atenție: nu commitați niciodată fișierul `.env` în repository-ul Git. Ironia construirii unui sistem de detecție a secretelor hardcodate ar fi completă dacă ați commita propriile chei API. Adăugați `.env` în `.gitignore` înainte de primul commit. Exemplu:

```

OPENAI_API_KEY=your-key-here
GITHUB_TOKEN=your-github-token-here

```

6 Sarcinile proiectului

Sistemul pe care îl construiți are un singur scop: un utilizator furnizează URL-ul unui repository și primește un raport de securitate și calitate structurat, cu

vulnerabilități clasificate, surse citate și sugestii de remediere. Sistemul devine mai bun pe măsură ce primește feedback.

Tot ce urmează descrie ce trebuie să existe la finalul fiecărei etape. Cum ajungeți acolo este treaba voastră.

6.1 Stratul de date: DTO-uri

Fișier: src/dtos.py

La finalul acestei etape: un fișier central cu toate structurile de date ale sistemului, validate cu Pydantic, importat de toți agenții. Niciun agent nu folosește dict simplu pentru a comunica cu altul.

Definiți:

- **Enumul** `VulnerabilitySeverity`: CRITIC, RIDICAT, MEDIU, SCAZUT, INFO
- **Enumul** `IssueType`: SECURITY_VULN, CODE_SMELL, STYLE_VIOLATION, DEPENDENCY_RISK, LOGIC_ERROR, HARDCODED_SECRET
- **Enumul** `Language`: PYTHON, JAVASCRIPT, TYPESCRIPT, JAVA, GO, OTHER
- **DTO-urile de parsare a codului**: `FunctionDTO` → `CodeFileDTO` → `RepositoryDTO`
- **DTO-urile de analiză**: `VulnerabilityDTO`, `CodeSmellDTO`, `DependencyRiskDTO`
- **DTO-urile de pipeline**: `RetrievalResultDTO`, `FileReviewDTO`, `FeedbackDTO`, `ReviewReportDTO`

Câmpurile complete ale fiecărui DTO se găsesc în secțiunea 8.

Ce evaluăm: toate semnăturile metodelor folosesc DTO-uri; un `ValidationError` la frontieră este semn că sistemul funcționează corect.

6.2 Corpus de securitate și indexare vectorială

Fișiere: src/tools/, scripts/build_index.py, scripts/evaluate_rag.py

La finalul acestei etape: un director `vectorstore/` persistent cu minimum 20 de documente de securitate indexate și un fișier `logs/rag_evaluation.json` cu scoruri $RAGAS \geq 0.6$ pe toate metricile.

- Colectați cel puțin 20 de documente din surse publice:
 - 8 CVE-uri din NVD (<https://nvd.nist.gov/developers/vulnerabilities>) via API REST – câte 2 per categorie din OWASP Top 10, selectate manual pentru relevanță față de Python/JavaScript. Filtrați la CVSS score ≥ 7.0 ;
 - 4 secțiuni din OWASP Top 10 2021 (<https://owasp.org/www-project-top-ten/>) – A01 la A04;
 - 3 intrări CWE relevante (<https://cwe.mitre.org/data/definitions/>) – CWE-89 (SQL Injection), CWE-79 (XSS), CWE-502 (Deserialization);
 - 3 secțiuni din documentația de securitate Django sau FastAPI (<https://docs.djangoproject.com/en/stable/topics/security/>, <https://fastapi.tiangolo.com/tutorial/security/>);

- 2 ghiduri de stil: PEP 8 (<https://peps.python.org/pep-0008/>) și Google Python Style Guide (<https://google.github.io/styleguide/pyguide.html>).

Sursa, data accesării și CVSS score (pentru CVE-uri) se documentează – fac parte din evaluare.

- Scrieți `load_corpus(corpus_dir)` care parcurge recursiv `corpus/` și returnează textul din fiecare document cu metadata: `source`, `doc_type`, `cvss_score` (pentru CVE), `owasp_category`.
- Scrieți `build_index(documents)` care face chunking, și embedding cu modelul `text-embedding-3-small` și stochează în ChromaDB. Justificați parametrii de chunking în comentariile din cod – de ce ați ales dimensiunea respectivă și nu alta.
- Scrieți `scripts/build_index.py` care rulează **o singură dată** și construiește atât `vectorstore/` (corpus de securitate), cât și `memory/feedback_index/` (inițializat gol, populat ulterior de `SelfImprovingFeedbackAgent`).
- Scrieți `scripts/evaluate_rag.py` care rulează RAGAS pe 10 întrebări de securitate specifice corpusului vostru (de ex. *“Ce este SQL injection și cum se previne?”*, *“Care este CVSS score-ul CVE-ului pentru Log4Shell?”*) și salvează scorurile în `logs/rag_evaluation.json`.

Ce evaluăm: diversitatea și acoperirea corpusului; justificarea parametrilor de chunking; scoruri RAGAS raportate și comentate; documentarea surselor.

6.3 Code Parser Agent

Fișier: `src/agents/parser_agent.py`

La finalul acestei etape: clasa `CodeParserAgent` cu metoda `parse(repo_url) -> RepositoryDTO` funcțională pe cel puțin 2 repository-uri publice GitHub, cu output JSON persistent în `data/review_exemplu.json`.

- Clonează repository-ul cu `GitPython` sau `subprocess + git clone` în `data/repos/<repo_name>/`.
- Identifică fișierele relevante pentru analiză: exclude `node_modules/`, `.git/`, `venv/`, fișierele binare și fișierele de configurare non-cod. Includeți un `.gitignore-style` filter configurabil.
- Pentru fiecare fișier relevant, construiți un `CodeFileDTO`: extrage limbajul din extensie, numărul de linii, lista de funcții/metode (cu `ast` pentru Python, cu `regex` pentru alte limbaje), lista de importuri și dependențe externe.
- Extrage fișierele de dependențe dacă există: `requirements.txt`, `pyproject.toml`, `package.json` – acestea alimentează `DependencyRiskDTO` în etapa de securitate.
- Agentul nu crășează pe un repository cu fișiere neașteptate – loghează fișierele care nu au putut fi parsate și continuă.
- Testați cu `scripts/test_parser.py`: afișați numărul de fișiere parsate, limbajele detectate, totalul de funcții extrase și primele 3 `CodeFileDTO`-uri complete.

Ce evaluăm: robustețea la structuri de repository variate; calitatea filtrelor de excludere; corectitudinea extragerii structurii codului.

6.4 RAG Security Agent

Fișier: src/agents/security_agent.py

La finalul acestei etape: clasa RAGSecurityAgent cu metoda scan(file_dto, k=5) -> list[VulnerabilityDTO], un prag de relevanță justificat și un heatmap salvat în logs/retrieval_heatmap.png.

- Construiește interogarea de recuperare semantic din conținutul fișierului: nu trimiteți întregul fișier la vectorstore – extrageți o descriere semantică a pattern-urilor de securitate detectabile (prin AST sau keyword matching): “*Python, SQL query construction with f-string, user input, no sanitization*”.
- Returnează top-*k* chunk-uri ordonate descrescător după scorul de similaritate, cu metadatele complete (cvss_score, owasp_category).
- Aplică filtru de relevanță: chunk-urile sub threshold sunt eliminate. Dacă lista rămâne goală, returnează [] și marchează fișierul cu coverage_empty=True.
- Verifică versiunile dependențelor din DependencyRiskDTO față de CVE-urile din vectorstore: dacă o librărie din requirements.txt are o versiune afectată de un CVE indexat, raportează un VulnerabilityDTO cu tipul DEPENDENCY_RISK.
- Construiește promptul cu ChatPromptTemplate și gpt-4o-mini; parsati răspunsul cu JsonOutputParser direct în list[VulnerabilityDTO].
- Testați cu scripts/test_retrieval.py pe 5 fragmente de cod reprezentative: câte unul pentru SQL injection, hardcoded secret, dependență vulnerabilă, deserialization nesigură și misconfiguration.
- Generați heatmap-ul 5×5 cu matplotlib și salvați-l în logs/retrieval_heatmap.png.

Ce evaluăm: construcția interogărilor semantice din cod; verificarea dependențelor față de CVE-uri; identificarea a cel puțin unui fals pozitiv documentat cu explicație.

6.5 Code Quality Agent

Fișier: src/agents/quality_agent.py

La finalul acestei etape: clasa CodeQualityAgent cu metoda evaluate(file_dto) -> list[CodeSmellDTO], testată pe cel puțin 2 repository-uri, cu logs/vulnerability_distribution.png generat.

- Calculează complexitatea ciclomatică a fiecărei funcții cu biblioteca radon. Orice funcție cu complexitate > 10 generează un CodeSmellDTO cu smell_type="HIGH_COMPLEXITY".
- Detectează code smells clasice:
 - *Long Method* – funcții cu mai mult de 50 de linii;

- *Magic Numbers* – literalii numerici în expresii (nu în definiții);
 - *Dead Code* – funcții sau importuri neutilizate (cu vulture sau analiză AST);
 - *Duplicate Code* – blocuri de cod aproape identice (similaritate Levenshtein sau hash-uri de AST);
 - *God Class* – clase cu mai mult de 10 metode publice și mai mult de 200 de linii.
- Verifică respectarea convențiilor de stil cu `pycodestyle` (pentru Python) sau `eslint` (pentru JavaScript/TypeScript). Grupați violările pe categorii, nu le raportați individual linie cu linie – un fișier care violează PEP8 în 50 de locuri primește un singur `CodeSmellDTO` cu `count=50`.
 - Calculează un `quality_score` per fișier între 0 și 100, scăzând puncte pentru fiecare categorie de problemă detectată. Formula este la latitudinea voastră, dar trebuie documentată și justificată în cod.
 - Generați `logs/vulnerability_distribution.png`: un grafic cu distribuția tipurilor de probleme pe toate fișierele analizate (securitate vs. calitate, defalcat pe severitate).

Ce evaluăm: diversitatea metrice de calitate implementate; justificarea `quality_score`; utilitatea practică a raportului generat.

6.6 Self-Improving Feedback Agent

Fișier: `src/agents/feedback_agent.py`

La finalul acestei etape: clasa `SelfImprovingFeedbackAgent` cu metodele `store_feedback()` și `augment_context()`, cu dovada că un feedback stocat modifică comportamentul unui review ulterior pe cod similar.

Acesta este agentul cu cel mai mare potențial de diferențiere – un tool SAST convențional nu are memorie episodică. Implementarea corectă a acestui agent este elementul care transformă proiectul din exercițiu academic în prototip de produs real.

- **Stocarea feedbackului:** când un utilizator marchează un finding ca fals pozitiv sau adaugă un comentariu la un review, agentul persistă un `FeedbackDTO` în `memory/feedback_store.json` și vectorizează descrierea finding-ului + contextul de cod + verdictul uman în `memory/feedback_index/`.
- **Recuperarea feedbackului:** la analiza unui fișier nou, agentul caută semantic în `memory/feedback_index/` finding-uri similare din trecut. Dacă găsește feedbackuri relevante (`similaritate > memory_threshold`), le injectează în promptul agentului de securitate:

“Nota din review-urile anterioare: un pattern similar (cursor.execute cu parametru extern) a fost marcat ca fals pozitiv în contextul test_.py deoarece datele de input sunt fixture-uri fixe, nu input utilizator. Verificați dacă contextul este același.”*

- **Dovada de funcționare:** prezentați în notebook cel puțin un ciclu complet: (1) analiza unui fișier vulnerabil, (2) marcare finding ca fals pozitiv de către utilizator, (3) stocarea feedbackului, (4) re-analiza unui fișier similar – dovediți că finding-ul nu mai apare sau apare cu o notă de context adăugată.
- Agentul nu îmbunătățește greutățile LLM-ului (fine-tuning) – îmbunătățește **contextul** furnizat LLM-ului. Distincția este importantă și trebuie explicată în notebook.

Ce evaluăm: ciclul complet stocare-recuperare-augmentare demonstrat; calitatea formatului FeedbackDTO; înțelegerea conceptuală a diferenței față de fine-tuning.

6.7 Orchestrare LangGraph

Fișier: src/graph/workflow.py

La finalul acestei etape: un StateGraph funcțional care rulează întregul pipeline de la URL de repository la raport final, cu o buclă de feedback dovedită că funcționează, un logger per nod și diagrama grafului exportată în logs/workflow_graph.png.

Starea grafului (WorkflowState ca TypedDict) conține:

- repo_url: str
- repository: RepositoryDTO
- context_map: dict[str, list[RetrievalResultDTO]]
- security_map: dict[str, list[VulnerabilityDTO]]
- quality_map: dict[str, list[CodeSmellDTO]]
- feedback_context: dict[str, list[FeedbackDTO]]
- critical_alert: bool
- report: ReviewReportDTO
- report_path: str
- iteration: int

Nodurile grafului:

- parse_repo → augment_with_memory → security_scan → quality_check_node → coverage_check
- coverage_check: dacă mai mult de 30% din fișiere au coverage_empty=True și iteration < MAX_ITER, întoarce la security_scan cu k mai mare și prag mai permisiv; altfel avansează.
- flag_critical: setează critical_alert=True dacă există cel puțin un VulnerabilityDTO cu severity=CRITIC – nu blochează pipeline-ul.
- generate_report

Logging per nod în logs/run_<timestamp>.json: nod, durată, tokeni consumați, număr de fișiere procesate, număr de vulnerabilități detectate.

Ce evaluăm: bucla de feedback demonstrată pe un caz concret; nodul augment_with_memory funcțional; corectitudinea stării; diagrama grafului.

6.8 Interfață Streamlit, Docker și notebook de demonstrație

Fișiere: src/app.py, Dockerfile, docker-compose.yml, notebooks/demo_pipeline.ipynb

La finalul acestei etape: aplicația pornește cu docker compose up fără niciun pas manual, iar notebook-ul demonstrează end-to-end sistemul pe un repository real.

Streamlit – organizat în sidebar + zonă principală:

- Sidebar: câmp text pentru URL-ul repository-ului GitHub, buton de analiză, slidere pentru pragul de relevanță și pragul de memorie episodică, checkbox pentru activarea/dezactivarea agentului de feedback.
- Zona principală: bară de progres cu mesaj per agent și per fișier analizat (“*Par-sare repository...*”, “*Recuperare feedback din memorie...*”, “*Scanare securitate: auth.py (3 / 12)...*”); banner st.error dacă critical_alert=True (“**ATENȚIE: Vulnerabilitate CRITICĂ detectată. Revizuiți imediat înainte de deployment.**”); tabel de fișiere cu scor de securitate colorat (CRITIC: #ffe3e3, RIDICAT: #fff0e0, MEDIU: #fff9c4, SCĂZUT/INFO: #d3f9d8); expander per fișier cu lista vulnerabilităților, fragmentul de cod afectat, CVE/OWASP citat și sugestia de remediere; secțiune separată “**Feedback**”: un buton de “*Fals pozitiv*” per finding, care stochează feedbackul prin SelfImprovingFeedbackAgent; buton de descărcare raport Markdown.
- Folosiți st.session_state pentru a nu re-rula pipeline-ul la fiecare interacțiune și pentru a păstra feedbackul în sesiunea curentă.

Docker:

- python:3.11-slim, port 8501, chei API prin -env-file .env – **nu** hardcodate în imagine.
- docker-compose.yml: două volume persistente – ./vectorstore:/app/vectorstore și ./memory:/app/memory. Fără al doilea volum, memoria episodică a agentului de feedback se pierde la fiecare docker compose up.
- Criteriu de acceptanță: docker compose up pornește aplicația și restaurează memoria episodică existentă fără intervenție manuală.

Notebook (demo_pipeline.ipynb) – în ordine:

- Introducere: repository-ul ales, motivul alegerii (vulnerabilitate de securitate reală sau construită deliberat), așteptări față de sistem.
- Apelul grafului LangGraph + starea finală completă.
- Scoruri RAGAS comentate: ce scor este acceptabil pentru un sistem de detecție a vulnerabilităților?

- logs/retrieval_heatmap.png, logs/vulnerability_distribution.png, logs/workflow_graph.png incluse cu `IPython.display.Image`.
- **Demonstrația memoriei episodice:** ciclul complet de feedback – înainte și după – cu capturi din FeedbackDTO stocat și diferența observată în review-ul următor.
- Raportul final redat cu `IPython.display.Markdown`.
- Concluzie: timpi per nod, cost estimat în USD per agent, rata de fals-pozitive observată pe repository-ul de test și o limitare reală identificată.

Ce evaluăm: docker compose up fără intervenție; ciclul de feedback demonstrat cu date reale; concluziile bazate pe logs/.

7 Criterii de evaluare

- Corpus de securitate și indexare ChromaDB + evaluare RAGAS: **10p**
- Code Parser Agent funcțional cu output DTO structurat: **10p**
- RAG Security Agent cu verificare dependențe, filtru de relevanță și heatmap: **15p**
- Code Quality Agent cu metrici de complexitate și code smells: **15p**
- Self-Improving Feedback Agent cu ciclu stocare-recuperare-augmentare demonstrat: **20p**
- Orchestrare LangGraph cu buclă de feedback, logging și graf vizualizat: **15p**
- Interfață Streamlit funcțională + Docker cu volume persistente: **15p**

Pentru bonus:

- Eleganță în implementare, structură ordonată a codului, comentarii cu explicații proprii – redactate ca și cum le-ați prezenta unui junior care nu a mai văzut LangGraph.
- Integrare Semgrep: rulați regulile Semgrep open-source ca primă trecere rapidă de pattern matching (independent de RAG), iar RAG-ul augmentează rezultatele Semgrep cu context și sugestii de remediere.
- Verificare automată a versiunilor de dependențe cu pip-audit sau safety: comparați rezultatele lor cu ce a detectat RAG Security Agent.
- Analiză cantitativă a memoriei episodice: măsurați reducerea ratei de fals-pozitive după N feedbackuri stocate pe un set de test fix.
- Suport multi-limbaj real: demonstrați analiza unui repository cu fișiere Python și JavaScript/TypeScript în același pipeline.
- Orice funcționalitate care depășește cerințele și demonstrează gândire inginerască matură față de problemele reale ale DevSecOps.

8 Indicații de implementare

Această secțiune conține detalii tehnice de implementare, organizate pe componente. Nu sunt cerințe – sunt puncte de plecare.

Câmpurile DTO-urilor

Folosiți class `VulnerabilitySeverity(str, Enum)` – moștenirea din `str` permite serializarea directă în JSON.

- **FunctionDTO:** `name`, `start_line: int`, `end_line: int`, `params: list[str]`, `cyclomatic_complexity: Optional[float]`
- **CodeFileDTO:** `file_path`, `language: Language`, `content: str`, `lines_of_code: int`, `functions: list[FunctionDTO]`, `imports: list[str]`, `dependencies: list[str]`
- **RepositoryDTO:** `url`, `name`, `local_path`, `files: list[CodeFileDTO]`, `total_loc: int`, `languages: list[Language]`
- **VulnerabilityDTO:** `id` (ex. "SEC-001"), `issue_type: IssueType`, `title`, `description`, `severity: VulnerabilitySeverity`, `file_path`, `line_number: Optional[int]`, `affected_snippet: Optional[str]`, `cve_id: Optional[str]`, `owasp_category: Optional[str]`, `fix_suggestion: str`, `cited_source: str`, `coverage_empty: bool = False`
- **CodeSmellDTO:** `smell_type`, `description`, `file_path`, `line_start: int`, `line_end: int`, `cyclomatic_complexity: Optional[float]`, `fix_suggestion: str`
- **DependencyRiskDTO:** `package_name`, `installed_version`, `vulnerable_versions: list[str]`, `cve_id`, `cvss_score: float`
- **RetrievalResultDTO:** `text`, `source`, `score: float`, `doc_type: str`, `cvss_score: Optional[float]`
- **FileReviewDTO:** `file_path`, `vulnerabilities: list[VulnerabilityDTO]`, `code_smells: list[CodeSmellDTO]`, `quality_score: float`
- **FeedbackDTO:** `feedback_id`, `original_finding_id`, `file_path`, `code_snippet`, `agent_verdict: str`, `is_false_positive: bool`, `human_comment: str`, `timestamp: str`
- **ReviewReportDTO:** `repo_url`, `repo_name`, `total_files`, `reviewed_files`, `critical_count`, `high_count`, `medium_count`, `low_count`, `file_reviews: list[FileReviewDTO]`, `overall_security_score: float`, `executive_summary: str`

Corpus și indexare

- NVD API – exemplu de request pentru CVE-urile unui an specific:

```
1 import requests
2 resp = requests.get(
3     "https://services.nvd.nist.gov/rest/json/cves/2.0",
4     params={"keywordSearch": "SQL injection Python",
5            "cvssV3Severity": "CRITICAL",
```

```

6         "resultsPerPage": 10},
7         headers={"apiKey": os.getenv("NVD_API_KEY")}
8     )
9     cves = resp.json()["vulnerabilities"]

```

NVD are un rate limit de 5 request-uri / 30 secunde fără API key și 50 / 30 secunde cu API key gratuit. Înregistrați-vă la <https://nvd.nist.gov/developers/request-an-api-key>.

- CVE-urile au câmpuri structurate JSON: cveId, descriptions[].value, metrics.cvssMetricV31[].cvssData.baseScore. Extrageți și stocați aceste câmpuri ca metadata în ChromaDB, nu doar textul.
- Dacă rulați build_index.py de mai multe ori fără să ștergeți vectorstore/, ChromaDB adaugă duplicate. Adăugați verificare la start.
- Indexul de memorie episodică (memory/feedback_index/) trebuie construit cu un collection_name diferit față de vectorstore-ul principal. Evitați amestecarea CVE-urilor cu feedbackuri umane în aceeași colecție.

Code Parser Agent

- Modulul ast din Python standard library parsează cod Python în AST complet: ast.parse(source), ast.walk(tree) pentru a extrage FunctionDef, Import, ClassDef. Este gratuit și precis – nu aveți nevoie de LLM pentru parsare.
- Pentru JavaScript/TypeScript, esprima (Python binding) sau tree-sitter cu grammar-ul corespunzător oferă același tip de AST.
- GitPython simplifică operațiile Git:

```

1 from git import Repo
2 repo = Repo.clone_from(url, local_path,
3                       depth=1) # shallow clone

```

depth=1 accelerează semnificativ clonarea pentru repository-uri mari.

- Fișierele de dependențe au formate diferite: requirements.txt (linie per pachet cu versiune), pyproject.toml (TOML, secțiunea [tool.poetry.dependencies]), package.json (JSON). Suportați cel puțin requirements.txt și package.json.

RAG Security Agent

- Nu trimiteți întregul conținut al unui fișier la vectorstore – interogarea ar fi prea generică și ar recupera CVE-uri pentru vulnerabilități care nu există în fișier. Extrageți pattern-uri de risc cu AST sau regex ca primă trecere, apoi construiți interogări targetate per pattern.
- Hardcoded secrets: regex simplu funcționează bine ca primă trecere. LLM-ul validează dacă e un secret real sau un placeholder.
- Verificarea dependențelor față de CVE-uri: parsați requirements.txt cu packaging.version pentru compararea versiunilor, nu cu string matching:

```

1 from packaging.version import Version
2 installed = Version("3.1.0")
3 vulnerable = Version("3.2.0")
4 is_vulnerable = installed < vulnerable

```

- Salvați răspunsurile LLM în cache cu `langchain.cache.SQLiteCache` pentru a evita re-apelurile identice în timpul dezvoltării. Atenție: invalidați cache-ul dacă modificați promptul sau corpusul.

Code Quality Agent

- `radon cc src/ -s -a` calculează complexitatea ciclomatică per funcție cu clasele A–F (A = simplă, F = extrem de complexă). Pragul > 10 corespunde claselor D–F.
- `vulture src/` detectează cod mort (funcții definite dar niciodată apelate) cu un nivel de încredere 0–100%. Raportați numai rezultatele cu încredere $\geq 80\%$.
- Duplication detection: hash-urile de subarbori AST (tehnica *clone detection*) sunt mai precise decât similaritatea textuală. Alternativă simplă: `pylint's -duplicate-code` opțiunea.
- `quality_score` sugestie de formulă: $score = 100 - (n_{\text{CRITIC}} \times 25) - (n_{\text{HIGH}} \times 10) - (n_{\text{MED}} \times 5) - (n_{\text{LOW}} \times 1)$, clamped la $[0, 100]$. Documentați formula în cod și justificați coeficienții.

Self-Improving Feedback Agent

- Vectorizarea feedbackului: nu vectorizați întregul `FeedbackDTO` – vectorizați o reprezentare semantică: “[FALS POZITIV] SQL query cu input extern în context de test, date sunt fixture fixe, nu input real utilizator”. Această reprezentare este recuperată la review-uri viitoare pe cod similar.
- Pragul `memory_threshold` pentru recuperarea feedbackului trebuie să fie mai ridicat decât threshold-ul pentru corpus-ul de securitate – un feedback recuperat greșit poate suprima o vulnerabilitate reală.
- Agentul nu elimină finding-uri bazat pe feedback – le **contextualizează**. Outputul este: “Vulnerabilitate posibilă. Nota din memorie: un pattern similar a fost fals pozitiv în test /. Verificați dacă contextul este același.” Decizia finală aparține reviewerului uman.
- Implementați o comandă de *reset* al memoriei episodice – utilă în evaluare pentru a demonstra comportamentul înainte și după feedback.

LangGraph

- La a doua trecere prin `security_scan` (după feedback loop), ajustați parametrii: k mai mare sau threshold mai mic. Detectați iterația din `state["iteration"]`.
- Fișierele dintr-un repository pot fi procesate în paralel cu `Send()`:

```

1 from langgraph.constants import Send
2 def dispatch_files(state):
3     return [Send("security_scan",
4                 {"file": f, "context": state["context_map"]})
5             for f in state["repository"].files]

```

Atenție: ChromaDB local nu garantează thread-safety complet la scrieri simultane – testați cu atenție dacă scrieți în vectorstore din noduri paralele.

- Export diagramă:

```

1 png = graph.compile().get_graph().draw_mermaid_png()
2 with open("logs/workflow_graph.png", "wb") as f:
3     f.write(png)

```

Streamlit și Docker

- Clonarea unui repository mare poate dura 30–60 de secunde. Afișați mesaje intermediare cu `st.status()` și faceți clonarea cu `depth=1` (shallow clone) pentru a reduce timpul.
- Butonul de feedback ("*Fals pozitiv*") trebuie să fie vizibil per finding, nu global. Folosiți `st.button(f"Fals pozitiv {finding.id}")` cu cheie unică per finding.
- Cele două volume Docker (vectorstore și memory) sunt ambele critice: fără memory, sistemul uită tot feedbackul la restart.
- `.env` și `data/repos/` în `.gitignore` înainte de primul commit – nu commitați repository-urile clonate local.

Resurse utile

- LangGraph: <https://langchain-ai.github.io/langgraph/>
- RAGAS: <https://docs.ragas.io>
- ChromaDB: <https://www.trychroma.com/>
- OpenAI API: <https://platform.openai.com/docs>
- NVD API (CVE database): <https://nvd.nist.gov/developers/vulnerabilities>
- NVD API key: <https://nvd.nist.gov/developers/request-an-api-key>
- OWASP Top 10 (2021): <https://owasp.org/www-project-top-ten/>
- OWASP ASVS: <https://owasp.org/www-project-application-security-verification-standard/>
- CWE Top 25: https://cwe.mitre.org/top25/archive/2024/2024_cwe_top25.html
- Django Security: <https://docs.djangoproject.com/en/stable/topics/security/>
- FastAPI Security: <https://fastapi.tiangolo.com/tutorial/security/>

- PEP 8: <https://peps.python.org/pep-0008/>
- Radon (cyclomatic complexity): <https://radon.readthedocs.io/>
- Semgrep (open-source rules): <https://semgrep.dev/r>
- GitPython: <https://gitpython.readthedocs.io/>
- Self-consistency (Wang et al., 2022): <https://arxiv.org/abs/2203.11171>
- MemGPT / memory-augmented LLMs (Packer et al., 2023): <https://arxiv.org/abs/2310.08560>